

API операционных систем

Потоки и синхронизация

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Общая информация

Основные положения

- По аналогии с процессами потоки выполнения представляют собой механизм для одновременного выполнения нескольких параллельных задач в рамках одного приложения
- При этом один процесс может содержать несколько потоков
- Все потоки выполняются внутри одной программы независимо друг от друга, разделяя общую глобальную память — в том числе данные и сегменты кучи

Основные положения

- Потоки в процессе могут выполняться одновременно
- В многопроцессорных системах возможно параллельное выполнение потоков
- Если же количество процессоров меньше количества потоков (в современных системах это почти всегда так), используется псевдопараллелизм – программы всех потоков бьются на элементарные (**атомарные**) инструкции, и процессор выполняет по небольшому набору инструкций от каждой в порядке очереди
- Очерёдность выполнения программ процессором определяется ОС и для программиста представляется как случайная

Преимущества и недостатки

- Потоки имеют множество преимуществ перед процессами
- В основном, эти преимущества связаны с тем, что потоки могут проще общаться между собой (благодаря использованию одной и той же памяти), а операции по работе с ними выполняются быстрее
- Однако следует помнить, что, в отличие от потоков, процессы более изолированы друг от друга, следовательно, крах одного не приводит к краху остальных
- Также процессы могут выполняться на различных физических машинах, что полезно, например, при запуске высокопараллельных программ на вычислительном кластере

Linux

Основные положения

- Для работы с потоками в Linux используется библиотека Pthread (потоки стандарта POSIX)
- Чтобы работать с данной библиотекой, необходимо подключить заголовочный файл `pthread.h`
- Названия функций и типов данной библиотеки начинается со слова `pthread`
- Чтобы пользоваться функциями данной библиотеки необходимо также прилинковать статическую библиотеку `pthread` (например, указав ключ `-lpthread` для `gcc` или `g++`)
- Функции библиотеки `pthread` подчиняются другому соглашению об ошибках: они возвращают 0 при успешном выполнении, а при возникновении ошибки возвращают её положительный код

Функции для работы с потоками

- `pthread_create` – создаёт и сразу запускает поток, выполняющий переданную функцию с переданным набором аргументов
- `pthread_exit` – вызывается из потока и завершает его работу с переданным кодом завершения
- `pthread_join` – “присоединяется” к переданному потоку, т.е. ждёт его завершения
- `pthread_detach` – “отсоединяет” поток, т.е. уведомляет его о том, что он может завершиться самостоятельно

Создание нового потока

- Создание нового потока производится функцией `pthread_create`, которая принимает следующие аргументы:
 - `__newthread` – указатель на **инициализированную** переменную типа `pthread_t`, в которую будет помещён идентификатор созданного потока
 - `__attr` – указатель на структуру с атрибутами потока (в рамках данного занятия будем оставлять `NULL` для установки значений по умолчанию)
 - `__start_routine` – указатель на функцию, принимающую один аргумент типа `void *` и возвращающую значение типа `void *` – именно она будет выполняться в потоке
 - `__arg` – указатель на `void`, с помощью которого в выполняемую в потоке функцию может быть передан аргумент любого типа
- После вызова функции сразу запускается новый поток, выполняющий переданную функцию параллельно с основной программой

Передача аргументов в поток

- Как видно из предыдущего слайда, в поточную функцию можно передать всего один указатель, тогда как многие функции, используемые в разработки, принимают гораздо большее количество аргументов
- Чтобы передать несколько аргументов функции, следует создать структуру Си (`struct`), поля которой будут содержать необходимые аргументы, и передавать в качестве аргумента поточной функции указатель на эту структуру
- Пример на следующем слайде демонстрирует, как передать в функцию два аргумента типов `int` и `double`. Поточковая функция преобразует аргумент к нужному типу, получает значения и выводит их на экран.

```
struct Arg
{
    int x;
    double y;
};

int main()
{
    struct Arg arg;
    arg.x = 5;
    arg.y = 5.5;

    pthread_t thread;
    pthread_create(&thread, NULL, f, &arg);
    /* ...дальнейшая работа... */
}
```

```
void * f(void * arg)
{
    struct Arg * arguments = (struct Arg *) arg;
    int x = arguments->x;
    double y = arguments->y;
    printf("%d, %f\n", x, y);
    /* ...дальнейшая работа... */

    pthread_exit(0);
}
```

Возврат значения и ожидание потока

- Если основной поток должен получить данные из порождённого, ему следует дождаться завершения порождённого потока с помощью функции `pthread_join`. Она принимает следующие аргументы:
 - `__th` – идентификатор потока, которого необходимо дождаться (ждать любого потока, по аналогии с `wait`, нельзя)
 - `__thread_return` – двойной указатель на `void`, в который будет помещено значение типа `void *`, содержащее значение, возвращённое потоком с помощью `pthread_exit`
- В примере на следующем слайде показано, как получить из потока возвращаемое значение типа `int`

```
void * f(void * arg)
{
    /* ...некоторая работа... */
    int * ret = malloc(sizeof(int));
    *ret = 5;
    pthread_exit((void *) ret);
}

int main()
{
    /* ...создание потока thread... */
    void * retVal;
    pthread_join(thread, &retVal);
    printf("%d\n", *((int *)retVal));
    free(retVal);
    return 0;
}
```

Отсоединение потока

- Если основному потоку не нужно получать данные, возвращённые функцией порождённого потока, а также не нужно выполнять других действий по его завершении, порождённый поток можно отсоединить функцией `pthread_detach`, принимающей в качестве аргумента идентификатор потока
- После вызова `pthread_detach` для потока к нему уже нельзя будет присоединиться с помощью `pthread_join`
- Необходимо учитывать, что при отсоединении потока всегда есть вероятность того, что основной поток завершится раньше, чем порождённый. В таком случае порождённый поток будет завершён аварийно.

Синхронизация

Проблемы многопоточности

- С помощью `pthread_create` можно создать более одного потока, выполняющего одну и ту же функцию
- Распространены ситуации, когда функциям, выполняемым в потоке, необходимо получать доступ к одной и той же переменной (объявленной глобально или доступной по указателю)
- Могут возникать ситуации, когда результат работы функций будет зависеть от очередности выполнения операций обработки разделяемых данных разными потоками, а ввиду того что эту очередность нельзя предсказать заранее (планировщик в ОС определяет это во время выполнения), результат работы функции также становится неопределённым. Такая ситуация называется **состоянием гонки**.

Пусть двум потокам необходимо выполнить следующую строку кода:

```
gVar = (gVar * 2) + (gVar % 3);
```

Время	Поток 1	Поток 2	Значение gVar
1	Расчёт gVar * 2 = 10		5
2		Расчёт gVar * 2 = 10	5
3		Расчёт gVar % 3 = 2	5
4		Расчёт 10 + 2 = 12	5
5		Присвоение gVar = 12	12
6	Расчёт gVar % 3 = 0		12
7	Расчёт 10 + 0 = 10		12
8	Присвоение gVar = 10		10 – некорректно!

Задача синхронизации

- Участки кода, в которых производится доступ к разделяемым ресурсам, называется критическим
- Чтобы избежать состояния гонки, необходимо обеспечить такие условия, при которых в каждый момент времени критический участок кода может выполняться только одним потоком
- Для этого используются примитивы синхронизации
- Простейшим примитивом синхронизации является мьютекс – объект ОС, который может находиться в двух состояниях – открытом и закрытом. При этом, когда один поток закрывает мьютекс, остальные не могут этого сделать до тех пор, пока первый его не откроет. Таким образом, мьютекс позволяет обеспечивать взаимное исключение выполнения критических участков кода.

Работа с мьютексами в Linux

- Для манипулирования мьютексами в Linux используются следующие функции:
 - `pthread_mutex_lock` – закрывает мьютекс, если он открыт. Если же мьютекс уже был закрыт другим потоком, ждёт, пока этот поток не откроет мьютекс.
 - `pthread_mutex_unlock` – открывает закрытый ранее этим потоком мьютекс
- При глобальном объявлении мьютекса рекомендуется использовать спецификатор `static`
- Для инициализации статически объявленного мьютекса должна использоваться константа `PTHREAD_MUTEX_INITIALIZER`
- В примере на следующем слайде показано, как можно применять мьютексы для защиты доступа к глобальной переменной

```
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int gVar = 5;

void * f(void * arg)
{
    pthread_mutex_lock(&mutex);

    gVar = (gVar * 2) + (gVar % 3);

    pthread_mutex_unlock(&mutex);

    pthread_exit(0);
}
```

Windows

Функции для работы с потоками

- **CreateThread** – создаёт поток, выполняющий переданную функцию с переданным набором аргументов, возвращает идентификатор и дескриптор потока
- **ExitThread** – вызывается из потока и завершает его работу с переданным кодом завершения
- **WaitForSingleObject (MultipleObjects)** – ожидает завершения потока
- **GetExitCodeThread** – получает код завершения, переданный из потока с помощью **ExitThread**

Создание нового потока

- Создание нового потока производится функцией **CreateThread**, которая принимает следующие аргументы:
 - **lpThreadAttributes** – указатель на структуру безопасности, в нашем случае будет устанавливаться в значение по умолчанию с помощью **NULL**
 - **dwStackSize** – размер стека для создаваемого потока, можно использовать 0, чтобы установить значение по умолчанию
 - **lpStartAddress** – указатель на функцию, принимающую один аргумент типа **LPCVOID** и возвращающую значение типа **DWORD** – именно она будет выполняться в потоке
 - **lpParameter** – переменная типа **LPCVOID**, содержащая указатель на любой элемент данных, который будет передан функции потока в качестве аргумента
 - **dwCreationFlags** – флаги создания потока, позволяющие отложить запуск потока
 - **lpThreadId** – указатель на переменную типа **DWORD**, в которую будет помещён идентификатор созданного потока
- Функция возвращает дескриптор нового потока в случае успешного завершения и значение **NULL** в случае ошибки

Отложенный запуск потока

- В отличие от Linux, Windows позволяет не запускать создаваемый поток сразу
- Это можно сделать, передав в аргумент `dwCreationFlags` значение `CREATE_SUSPENDED`
- В таком случае, чтобы всё-таки запустить выполнение потока, необходимо будет вызвать функцию `ResumeThread`, передав ей дескриптор созданного потока
- В примере кода на следующем слайде показано, как запустить функцию на выполнение в потоке, отложив её запуск, а также дождаться завершения потока и получить код завершения

```
DWORD WINAPI f(LPVOID arg){
    printf("Hello from thread!\n");
    ExitThread(0);
}

int main(int argc, char *argv[]){
    DWORD threadId = 0;
    DWORD arg = 5;
    HANDLE hThread = CreateThread(NULL, 0, f, (LPVOID) &arg,
CREATE_SUSPENDED, &threadId); // Не нужно так делать в практических!
    if (hThread == NULL){
        printf("Error: %d\n", GetLastError());
        return 1;
    }
    if (ResumeThread(hThread) == -1){
        printf("Error: %d\n", GetLastError());
        return 1;
    }
    WaitForSingleObject(hThread, INFINITE);
    DWORD exitCode = 0;
    GetExitCodeThread(hThread, &exitCode);
    return 0;
}
```

Синхронизация

Работа с критическими секциями в Windows

- Windows предоставляет простое средство работы с критическими секциями кода – объекты `CRITICAL_SECTION`
- Объекты данного типа позволяют отмечать критические секции в коде и обеспечивать их эксклюзивное выполнение
- Чтобы работать с объектом `CRITICAL_SECTION`, его необходимо инициализировать с помощью функции `InitializeCriticalSection`, принимающей указатель на инициализируемый объект
- Далее, при входе в критическую секцию, поток должен вызывать функцию `EnterCriticalSection`, а при выходе из неё – `LeaveCriticalSection`, передавая им указатели на соответствующий объект `CRITICAL_SECTION`
- По окончании использования объект должен быть удалён с помощью `DeleteCriticalSection`

```
CRITICAL_SECTION cs;
```

```
int gVar = 5;
```

```
DWORD WINAPI f(LPVOID arg)
```

```
{
```

```
    EnterCriticalSection(&cs);
```

```
    gVar = (gVar * 2) + (gVar % 3);
```

```
    LeaveCriticalSection(&cs);
```

```
    ExitThread(0);
```

```
}
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    InitializeCriticalSection(&cs);
```

```
    /* ...запуск и ожидание завершения потоков... */
```

```
    DeleteCriticalSection(&cs);
```

Работа с мьютексами в Windows

- Как и в Linux, в Windows предоставляется возможность работы с мьютексами, однако в данной ОС они могут выполнять более расширенный круг функций. Для простой защиты разделяемых данных в рамках одного процесса следует использовать критические секции.
- Мьютекс, в отличие от `CRITICAL_SECTION`, является объектом ядра, а значит, доступ к нему осуществляется с помощью дескриптора (`HANDLE`), а ожидание производится с помощью `WaitForSingleObject`
- Не существует отдельной функции для захвата мьютекса в Windows: этот функционал возложен на `WaitForSingleObject`, которая, дождавшись освобождения мьютекса (если тот был занят), захватывает его

Функции для работы с мьютексами в Windows

- Чтобы создать мьютекс используется функция **CreateMutex**, принимающая следующие аргументы:
 - **lpMutexAttributes** – атрибуты безопасности, оставляемые по умолчанию с помощью **NULL**
 - **bInitialOwner** – аргумент типа **BOOL**, позволяющий потоку, создающему мьютекс, захватить его сразу после создания
 - **lpName** – имя мьютекса (для использования из других процессов), может быть оставлено в **NULL**, при этом мьютекс останется безымянным
- Функция возвращает **HANDLE** для созданного мьютекса в случае успешного завершения и **NULL** в случае ошибки
- Для освобождения захваченного мьютекса используется функция **ReleaseMutex**, принимающая дескриптор захваченного мьютекса
- В примере на следующем слайде показано, как защитить те же разделяемые данные с помощью мьютекса

```
HANDLE hMutex = NULL;
```

```
int galVar = 5;
```

```
DWORD WINAPI f(LPVOID arg)
```

```
{  
    WaitForSingleObject(hMutex, INFINITE);  
    galVar = (galVar * 2) + (galVar % 3);  
    ReleaseMutex(hMutex);  
    ExitThread(0);  
}
```

```
int main(int argc, char *argv[])
```

```
{  
    hMutex = CreateMutex(NULL, FALSE, NULL);  
    /* ...запуск и ожидание завершения потоков... */  
    CloseHandle(hMutex);  
}
```


Отличия мьютексов и критических секций в Windows

Как видно из примера выше, использование мьютексов для защиты разделяемых переменных не несёт в себе каких-либо принципиальных отличий от использования критических секций для той же цели. В связи с этим необходимо выделить такие отличия для общего случая:

- Благодаря тому, что мьютексы являются поименованными (в общем случае) объектами ядра, они могут быть использованы для синхронизации не только потоков, но и процессов, например, при совместном доступе к файлу
- Для мьютексов имеется возможность ожидания в течение некоторого интервала времени, чего не предусмотрено для критических секций

Отличия мьютексов и критических секций в Windows

- Для мьютекса, в отличие от объекта критической секции, предоставляется возможность захвата сразу при создании
- Когда поток завершается, так и за освободив занятый мьютекс, мьютекс освобождается автоматически (в таком случае `WaitForSingleObject` вернёт особое значение `WAIT_ABANDONED`). В случае с критическими секциями, если поток выполнил вход, но завершился, так и не выполнив выход, объект секции разблокирован не будет, что приведёт к ожиданию её другими потоками в течение неопределённого времени.
- При всех описанных недостатках функциональности объекты `CRITICAL_SECTION`, как правило более производительны, поэтому их использование предпочтительно там, где нет необходимости пользоваться преимуществами мьютексов

Полезная литература

- Майкл Керриск – Linux API. Исчерпывающее руководство. – Книга, на основании которой сделана данная презентация. Для закрепления материала рекомендуются главы 29 и 30. Для более углублённого ознакомления – главы 31, 32 и 33.
- Джонсон М. Харт. Системное программирование в среде Windows. – Книга, на основании которой сделана данная презентация. Для закрепления материала рекомендуются главы 7 и 8. Для более углублённого ознакомления – главы 9, 10.